The heteroTests Package: A Unified Toolkit for Heteroscedasticity Diagnostics

by Diogo Ribeiro

Abstract Heteroscedasticity diagnostics in R are spread across a dozen packages whose function names, argument orders and return objects do not agree, so combining several tests in one analysis requires bridging code. **heteroTests** implements 41 diagnostics—auxiliary-regression, group-comparison, rank-based, nonconstant-variance, ARCH-type, and resampling-based—behind one interface. Each `perform*Test()` returns an S3 `htest` object carrying the choices that determine the statistic (residual type, trimming, lag order, grouping), which is what makes batch execution through `runHeteroTests()` and automated reporting possible. The package imports only base R and **MASS**, using **car** and **sandwich** when installed. Timings scale linearly to $n = 50\,000$, where twelve of the thirteen benchmarked tests run in under 500 ms and White's takes 634 ms, and simulated power exceeds 80% against continuous, step-function and autoregressive variance patterns. Where an established implementation exists we reproduce its statistic to machine precision; where one does not, the implementation is checked against its published definition. The package is distributed on GitHub under Apache-2.0.

1 Introduction

Heteroscedasticity tests in R reside in **lmtest**, **car**, and base **stats** yet use distinct function names, argument orders, and output formats. Users call `bptest()`, `ncvTest()`, and `spreadLevelPlot()`, then write custom code to align outputs and metadata. This process increases boilerplate code and complicates reproducible workflows.

heteroTests places 41 diagnostics behind one API. Each `perform*Test()` returns an S3 `htest` object recording the residual type, trimming, lag selection and grouping used, so a result can be reproduced from the object alone; `runHeteroTests()` executes a configured set in one call.

The package imports base R and **MASS**, and uses **car** and **sandwich** when they are installed. Timings scale linearly to $n = 50\,000$, where all but one benchmarked test run in under 500 ms, and simulated power exceeds 80% against linear, step-function and autoregressive variance patterns.

Section 2.2 reviews statistical tests; Section 2.3 describes package architecture and API; Section 2.4 presents usage examples and case studies; Section 2.5 provides simulation results and benchmarks; Section 2.6 compares with existing packages; Section 2.7 discusses best practices; and Section 2.8 outlines future developments.

Motivation and Problem Statement

The fragmentation of heteroscedasticity testing across multiple R packages creates several challenges for practitioners:

1. **Inconsistent interfaces**: Different packages use varying function names, argument structures, and return formats. For example, **lmtest** uses `bptest()` for the Breusch-Pagan test, while **car** provides `ncvTest()` for non-constant variance testing.
2. **Package dependencies**: Users must load multiple packages and manage their dependencies, often encountering namespace conflicts or version incompatibilities.
3. **Output harmonization**: Results require manual standardization for comparative analysis. Each package returns different object structures, making batch processing cumbersome.
4. **Workflow complexity**: Integrating multiple tests into analysis pipelines requires substantial boilerplate code for input validation, result extraction, and formatting.

The **heteroTests** package addresses these issues by providing a unified interface that:

- Standardizes all test functions under the `perform*Test()` naming convention
- Returns consistent S3 `htest` objects with enhanced metadata
- Enables batch execution through `runHeteroTests()`
- Maintains minimal dependencies while supporting optional enhancements
- Scales efficiently to large datasets with linear time complexity

Package Overview

heteroTests exports 41 `perform*Test()` diagnostics in seven groups:

Auxiliary regression tests Model squared residuals on covariates (White, Breusch-Pagan, Koenker, Harvey, Park, Glejser)

Group-wise comparison tests Compare variances across data partitions (Goldfeld-Quandt, Levene, Brown-Forsythe, Bartlett)

Rank-based methods Use nonparametric rank statistics (Spearman rank correlation, rank permutation)

Non-constant variance diagnostics Simple regression-based variance trend detection (Cook-Weisberg NCV, Moses spread-level)

ARCH-type tests Time-series conditional heteroscedasticity (Engle ARCH LM, McLeod-Li)

Resampling and robust diagnostics Procedures that avoid the asymptotic null (wild bootstrap, quantile-regression, rank-permutation, high-dimensional)

Specialized tests Domain-specific diagnostics (spatial, clustered, extreme ratios)

The argument conventions and the `htest` return structure are common to all of them, which is what allows the batch runner to treat them interchangeably. Three further components are described in Sections 2.3 and 2.7: a batch runner with result caching (`runHeteroTests()`), streaming implementations of the auxiliary-regression tests for data that do not fit in memory, and a recommendation layer that interprets the diagnostics and proposes corrections such as weighted least squares.

2 Statistical Background and Notation

We consider the linear regression model

$$y = X\beta + \varepsilon, \quad E(\varepsilon) = 0,\ \mathrm{Var}(\varepsilon) = \Sigma = \mathrm{diag}(\sigma_1^2, \ldots, \sigma_n^2),$$

where $\hat{\beta} = (X^\top X)^{-1} X^\top y$, $\hat{y} = X\hat{\beta}$, and residuals are $e = y - \hat{y}$. Under homoscedasticity, $\sigma_i^2 = \sigma^2$; under heteroscedasticity, the σ_i^2 vary across observations. All diagnostics in **heteroTests** operate on e or transformations of e, with the user selecting `resid.type` (response, Pearson, or studentized).

Table 1 lists six of the seven groups with a representative wrapper; the resampling diagnostics of Section 2.2.8 are omitted from the table because they share the auxiliary-regression statistics and differ only in how the null is computed. The wrapper `runHeteroTests()` accepts arguments `model`, `data`, `resid.type`, `trim`, `lag`, `group`, and `order.by`, dispatching to the selected diagnostics and returning a named list of S3 `htest` objects.

Table 1: Categories of heteroscedasticity tests and example wrappers

Category	Wrapper pattern	Example
Auxiliary-regression	`perform<White|BP|Koenker>Test()`	`performWhiteTest`
Group-comparison	`perform<Levene|BrownForsythe>Test()`	`performLeveneTest`
Rank-based	`perform<Spearman|RankPermutation>Test()`	`performSpearmanTest`
Non-constant variance	`performNCVTest()`	`performNCVTest`
ARCH-type	`perform<ARCHLM|McLeodLi>Test()`	`performARCHLMTest`
Specialized	`perform<Pesaran|OBrien>Test()`	`performPesaranTest`

Classification of Heteroscedasticity Tests

The categories below group tests by the statistic they compute and the assumptions it requires.

Auxiliary Regression Tests Fit an auxiliary regression of squared residuals (or functions thereof) on covariates, detecting general variance patterns. Examples include White's test, Breusch–Pagan test, Koenker's studentized version, Harvey's multiplicative test, Park's logarithmic test, and Glejser's absolute residual test.

Group-wise Comparison Tests Partition data by ordered covariates or factor levels, then compare sample variances with F- or Chi-square statistics. Examples include Goldfeld–Quandt test, Levene's test, Brown–Forsythe test, Bartlett's test, Fligner–Killeen test, and Hartley's F_{max} test.

Rank-Based Methods Transform absolute residuals via ranks and apply nonparametric tests (correlation or rank-sum) to detect variance trends. Examples include Spearman's rank correlation test and the rank-permutation test.

Non-Constant Variance Diagnostics Use graphical and simple regression diagnostics to assess monotonic variance patterns without full auxiliary regressions. Examples include Cook–Weisberg NCV test and Moses spread–level plot.

ARCH-Type Tests For time-series data, test for serial dependence in squared residuals using autoregression or portmanteau statistics. Examples include Engle's ARCH LM test and McLeod–Li portmanteau test.

Specialized Tests Address specific variance structures such as spatial clustering, extreme variance ratios, or group-specific heteroscedasticity patterns. Examples include Pesaran's spatial heteroscedasticity test and O'Brien's scaled-deviation test.

Detailed descriptions of each category and their implementations appear in Sections 2.2.2 through 2.2.7.

Auxiliary Regression Tests

Auxiliary regression tests detect heteroscedasticity by modeling the squared residuals (or functions thereof) from the primary regression on explanatory variables or their transformations. Under the null hypothesis of homoscedasticity, the auxiliary regression has no explanatory power beyond the intercept. We summarize key variants below.

White's General Test White's test does not require specification of the heteroscedasticity form. Fit the auxiliary regression:

$$e_i^2 = \alpha_0 + X_i\alpha + Z_i\gamma + u_i, \tag{1}$$

where Z_i contains all distinct cross-products $X_{ij}X_{ik}$ and squares X_{ij}^2. The Lagrange multiplier statistic is

$$LM = nR_{aux}^2,$$

where R_{aux}^2 is the coefficient of determination. Under H_0, $LM \sim \chi_q^2$, with $q = \dim(\alpha, \gamma)$ (White, 1980). The test is consistent against general alternatives but may suffer power loss in small samples due to overfitting. In **heteroTests**, White's test is implemented via `performWhiteTest()`, which automatically constructs the auxiliary regression matrix including cross-terms and squares.

Breusch–Pagan Test Assumes variance is a linear function of regressors. Fit:

$$e_i^2 = \alpha_0 + X_i\alpha + v_i. \tag{2}$$

The LM statistic is

$$LM = \frac{nR_{aux}^2}{2},$$

where the denominator 2 reflects the assumption that v_i has variance $2\sigma^4$ under normality. Under H_0, $LM \sim \chi_{p-1}^2$ (Breusch and Pagan, 1979). The Breusch–Pagan test is more powerful when the true variance function is close to linear in the covariates. The `performBPTest()` function implements this via auxiliary regression of squared residuals on the original design matrix.

Koenker's Studentized BP Test A robust variant of Breusch–Pagan, replacing the normality-dependent scaling with a more general approach. The test statistic is computed as:

$$LM = nR_{aux}^2, \tag{3}$$

where R^2_{aux} comes from regressing squared residuals on regressors, but the asymptotic distribution remains χ^2_{p-1} without the factor of $1/2$ (Koenker, 1981). This version is less sensitive to non-normality of errors and is implemented in `performKoenkerTest()`.

Harvey's Test Models the log-variance as a linear function:

$$\log(e_i^2) = \alpha_0 + X_i\alpha + h_i,$$

and tests $H_0 : \alpha = 0$ via an F-statistic or LM approach. This formulation is suitable when multiplicative heteroscedasticity is expected, as $\mathrm{Var}(\varepsilon_i) = \exp(X_i\alpha)$ under the alternative (Harvey, 1976). The `performHarveyTest()` function handles the log-transformation automatically, with appropriate treatment of zero or negative squared residuals.

Park's Test Regresses log-squared residuals on the log of a single regressor:

$$\log(e_i^2) = \alpha_0 + \gamma \log(X_{ij}) + p_i.$$

Under H_0, $\gamma = 0$. The t-test on $\hat{\gamma}$ follows t_{n-p} asymptotically (Park, 1966). This test is particularly useful when theory suggests that variance scales as a power of a specific covariate. The `performParkTest()` function allows specification of which regressor to use in the auxiliary regression.

Glejser's Test Regresses absolute residuals on powers of covariates:

$$|e_i| = \alpha_0 + \sum_j \delta_j |X_{ij}|^{\kappa_j} + g_i,$$

with various choices of κ_j (e.g., 1, 1/2, 2). Each coefficient is tested individually via t-tests (Glejser, 1969). The `performGlejserTest()` function provides options for different power specifications and can test multiple functional forms simultaneously.

Group-wise Comparison Tests

Group-wise comparison tests split the data into two or more subsets and compare sample variances across these groups. Under homoscedasticity, group variances should be equal. Below, we detail key variants and their test statistics.

Goldfeld–Quandt Test Order observations by a covariate thought to influence variance. Omit c central cases, divide the remaining into two groups of sizes n_1 and n_2. For each group, fit the original model and compute residual sum of squares (RSS_1, RSS_2). The test statistic is

$$F = \frac{\mathrm{RSS}_1/(n_1 - p)}{\mathrm{RSS}_2/(n_2 - p)},$$

which under H_0 follows an $F_{n_1-p,\,n_2-p}$ distribution (Goldfeld and Quandt, 1965). Choosing c trades off bias and power; common practice is $c = \lfloor n/3 \rfloor$. The `performGoldfeldQuandtTest()` function allows custom specification of the ordering variable via the `order.by` argument and the fraction of central observations to omit.

Levene's Test Levene's test compares absolute deviations from group means. Define

$$z_{ij} = |y_{ij} - \bar{y}_j|,$$

where y_{ij} is observation i in group j, and $\bar{y}_j$ is the mean of group j. Perform one-way ANOVA on z_{ij}:

$$W = \frac{(N-g)\sum_j n_j(\bar{z}_j - \bar{z})^2}{(g-1)\sum_{j,i}(z_{ij} - \bar{z}_j)^2},$$

with N total observations and g groups. Under H_0, $W \sim F_{g-1,\,N-g}$ (Levene, 1960). Levene's test is robust to departures from normality. The `performLeveneTest()` function accepts formula notation for group specification and integrates with **car::leveneTest()** when available.

Brown–Forsythe Test A variant of Levene's test using group medians or trimmed means. Define

$$z_{ij} = |y_{ij} - \tilde{y}_j|,$$

where $\tilde{y}_j$ is the median (or trimmed mean) of group j. The ANOVA F statistic on z_{ij} follows $F_{g-1,\,N-g}$ under H_0 (Brown and Forsythe, 1974). It increases robustness against outliers. The `performBrownForsytheTest()` function implements both median-based and trimmed-mean variants, with trimming proportion controlled by the `trim` argument.

Bartlett's Test Assumes normally distributed errors. The test statistic is

$$\chi^2 = \frac{(N-g)\ln S^2 - \sum_j (n_j - 1)\ln S_j^2}{1 + \frac{1}{3(g-1)}\left(\sum_j \frac{1}{n_j - 1} - \frac{1}{N-g}\right)},$$

where S_j^2 is the sample variance in group j and S^2 is the pooled variance. Under H_0, the statistic approximately follows χ^2_{g-1} (Bartlett, 1937). Bartlett's test is powerful under normality but sensitive to non-normal distributions. The `performBartlettTest()` function provides this parametric approach with automatic degrees-of-freedom correction.

Fligner–Killeen Test A nonparametric test based on ranks of absolute deviations from medians. Compute ranks R_{ij} of $|y_{ij} - \tilde{y}_j|$ pooled across groups. The test statistic

$$\chi^2 = \frac{\sum_j n_j (\bar{R}_j - \bar{R})^2}{\sum_{j,i} (R_{ij} - \bar{R})^2 / (N-1)},$$

follows χ^2_{g-1} approximately, robust to non-normality (Fligner and Killeen, 1976). The `performFlignerKilleenTest()` function implements this rank-based approach with exact p-values for small samples.

Hartley's $F_{\max}$ Test Compares the ratio of the largest to smallest group variances:

$$F_{\max} = \frac{\max_j S_j^2}{\min_j S_j^2}.$$

Under H_0, critical values depend on g and n_j (Hartley, 1950). The test is sensitive to a single outlier variance but powerful when one group has substantially different variance. The `performHartleyTest()` function includes critical value tables for common sample sizes and group numbers.

Rank-Based Methods

Rank-based methods transform absolute residuals via ranks and apply nonparametric correlation or rank-sum tests to detect variance trends. These approaches are robust to non-normal errors while maintaining reasonable power.

Spearman Rank Correlation Test Compute the Spearman rank correlation ρ_S between $|e_i|$ and a covariate X_j (or fitted values $\hat{y}_i$):

$$t = \rho_S \sqrt{\frac{n-2}{1-\rho_S^2}} \sim t_{n-2}$$

under H_0 of no monotonic variance trend. The `performSpearmanTest()` function computes this correlation for each regressor and provides omnibus testing across all covariates.

Non-Constant Variance Diagnostics

Non-constant variance diagnostics use simple modeling and graphical tools to detect monotonic relationships between residual spread and fitted values without fitting high-dimensional auxiliary regressions. They are well-suited for rapid checks and transformation guidance.

Cook–Weisberg NCV Test Model the squared residuals against fitted values:

$$e_i^2 = \alpha_0 + \alpha_1 \hat{y}_i + v_i. \tag{4}$$

The null hypothesis $H_0 : \alpha_1 = 0$ is tested via the t-statistic for $\hat{\alpha}_1$, which under homoscedasticity follows a t_{n-p} distribution. A significant coefficient indicates a linear variance–mean trend (Cook and Weisberg, 1983). The `performNCVTest()` function provides this simple diagnostic with options for different variance stabilizing transformations.

Spread–Level Plot (Moses Test) Construct a plot of

$$(x_i, z_i) = (\log \hat{y}_i, \log |e_i|)$$

and overlay a smooth curve. Compute the Spearman rank correlation ρ between $\log |e_i|$ and $\log \hat{y}_i$; the test statistic

$$t = \rho \sqrt{\frac{n-2}{1-\rho^2}} \sim t_{n-2}$$

assesses monotonic variance changes. This approach combines visual diagnosis with a nonparametric test (Moses, 1964). The `performMosesTest()` function produces both the test statistic and diagnostic plot.

ARCH-Type Tests

ARCH-type tests evaluate conditional heteroscedasticity in time-series by examining the serial dependence of squared residuals. Under the null hypothesis of no ARCH effects, squared residuals should behave like white noise. Key procedures are detailed below.

Engle's ARCH LM Test Fit the original time-series model (e.g., ARIMA) and obtain residuals e_t. Regress squared residuals on L their own lags:

$$e_t^2 = \alpha_0 + \sum_{i=1}^{L} \alpha_i e_{t-i}^2 + u_t. \tag{5}$$

Compute the Lagrange multiplier statistic

$$LM = T \times R_{arch}^2,$$

where R_{arch}^2 is from the auxiliary regression and T is the number of observations used. Under H_0, $LM \sim \chi_L^2$ asymptotically (Engle, 1982). Choice of L can follow information criteria or graphical inspection of squared-residual autocorrelations. The `performARCHLMTest()` function allows flexible lag specification and automatic lag selection via information criteria.

McLeod–Li Test This portmanteau test applies the Ljung–Box statistic to squared residuals instead of raw residuals. Let $\hat{\rho}_k$ be the sample autocorrelation of e_t^2 at lag k. The test statistic is

$$Q = n(n+2) \sum_{k=1}^{L} \frac{\hat{\rho}_k^2}{n-k},$$

which under H_0 follows χ_L^2 for large n (McLeod and Li, 1978). Significant Q indicates ARCH effects. The McLeod–Li test is powerful against a broad range of nonlinearity in volatility. The `performMcLeodLiTest()` function implements this with automatic lag selection and small-sample corrections.

Other Specialized Tests

These tests target specific patterns of heteroscedasticity—spatial clustering, extreme variance ratios, or group-specific structures—that standard methods may not detect effectively.

Pesaran's Spatial Test Designed for cross-sectional data with spatial correlation in error variances. Compute spatially weighted residuals:

$$w_i = \sum_j w_{ij} e_j^2,$$

where w_{ij} are spatial weights (typically based on geographic distance or adjacency). The test statistic

$$S = \frac{e^\top W e}{e^\top e},$$

follows approximately a χ^2 distribution under H_0 of no spatial heteroscedasticity (Pesaran, 1997). The `performPesaranTest()` function accepts spatial weight matrices and provides both exact and asymptotic p-values.

O'Brien's Scaled Deviation Test Transforms observations to account for group-level variance heterogeneity. Define

$$t_{ij} = \frac{(N-1)(y_{ij} - \bar{y}_j)^2 - S_j^2(n_j - 1)}{(n_j - 1)S_j^2}$$

where S_j^2 is group j variance. Perform ANOVA on t_{ij}; the resulting F statistic tests equality of variances across groups robustly in non-normal settings (O'Brien, 1979). The `performOBrienTest()` function implements this transformation with optional trimming.

Modern Resampling and Robust Diagnostics

The classical statistics above rely on asymptotic χ^2 or F approximations that can be unreliable in small samples or under heavy-tailed errors. **heteroTests** therefore complements them with a family of resampling-based and heteroscedasticity-robust diagnostics that trade analytic tractability for finite-sample validity.

Wild Bootstrap Test Rather than relying on the asymptotic null distribution of the White or Breusch–Pagan statistic, the wild bootstrap regenerates the response as $y_i^* = \hat{y}_i + e_i\, v_i$, where the multipliers v_i are drawn from a distribution with $E(v_i) = 0$ and $E(v_i^2) = 1$ (the Rademacher and Mammen two-point schemes are supported). Recomputing the statistic over B such samples yields an empirical reference distribution and a bootstrap p-value $(1 + \#\{T_b^* \geq T\})/(B+1)$. The procedure preserves the conditional heteroscedasticity structure of the residuals and provides accurate size control even when n is small (Davidson and Flachaire, 2008). It is implemented in `performWildBootstrapTest()`.

Quantile-Regression Test Heteroscedasticity manifests as conditional quantiles of y that fan out or contract with the covariates. Fitting regression quantiles (Koenker and Bassett, 1978) at a low and a high τ (e.g. 0.25 and 0.75) and testing the equality of the corresponding slope vectors detects exactly this divergence, without committing to a parametric variance function. The `performQuantileRegressionTest()` function implements the comparison, delegating the quantile fits to **quantreg** when available.

Rank-Permutation, High-Dimensional, and Spatial Tests Three further diagnostics address settings where the standard asymptotics break down. `performRankPermutationTest()` replaces the analytic null of a rank-based variance statistic with a permutation distribution, giving exact inference for small samples. `performHighDimensionalTest()` targets the $p \approx n$ regime in which the auxiliary regression is ill-conditioned, using a regularized score statistic. `performSpatialHeteroTest()` extends variance testing to spatially correlated errors, taking a spatial-weights object (`listw`) and assessing whether residual variance clusters in space.

3 Package Design and Implementation

Architecture

The **heteroTests** package follows a modular, maintainable structure, organized as follows:

- **R/**: Contains user-facing functions. Each file corresponds to a test wrapper `perform*Test()`, e.g.,:
  - `performWhiteTest.R`: Input validation, extraction of residuals, call to `whiteTestCore()`.
  - `performLeveneTest.R`: Formula parsing, grouping logic, fallback to `car::leveneTest()` if available.
- **tests/**: Core implementations and helper utilities:
  - `whiteTestCore.R`: Low-level auxiliary regression code.
  - `utils.R`: Common functions for residual extraction, model matrix handling, and error messaging.
- **data/**: Example datasets included for illustrations:
  - `hetero_data.rda`: Simulated data exhibiting heteroscedasticity.
- **man/**: Roxygen2-generated documentation files (`.Rd`) for each exported function and dataset.
- **vignettes/**: Workflow tutorials and examples in R Markdown (e.g., `tutorial.Rmd`, `using_heteroTests.Rmd`).
- **inst/**: Includes package assets such as logos and CSS for pkgdown site.

All test wrappers return an S3 object of class `htest` enriched with:

- `statistic`, `parameter`, `p.value`, `method`, `data.name` (standard htest fields).
- Additional metadata: `resid.type`, `lag`, `trim`, `group`, or `order.by` as appropriate.

This separation of interface (`R/`) and implementation (`tests/`) ensures ease of extension and isolated unit testing.

API Conventions

The **heteroTests** package offers a uniform set of functions for running heteroscedasticity diagnostics, all following the `perform*Test()` naming scheme and standardized argument structure.

Function Naming Each diagnostic is exposed via a wrapper named: `perform<MethodName>Test()` e.g., `performWhiteTest()`, `performLeveneTest()`, `performARCHLMTest()`.

Core Arguments

- `model`: A fitted model object (classes: `lm`, `glm`, `Arima`).
- `data`: Optional data frame; only required if the model was fit with a formula and data.

Control Parameters All tests accept a consistent set of optional parameters:

- `resid.type`: Type of residuals (`"response"`, `"pearson"`, `"student"`).
- `trim`: Proportion to trim in group tests (numeric, $0 \leq$ `trim` < 0.5).
- `lag`: Number of lags for ARCH-type tests (integer ≥ 1).
- `order.by`: Numeric vector or formula to define ordering in Goldfeld–Quandt.

Return Object Each `perform*Test()` returns an S3 object of class `htest` with fields:

- `statistic`, `parameter`, `p.value`, `method`, `data.name`.
- Supplementary metadata in list slots, named after control parameters (e.g., `resid.type`, `lag`, `trim`).

Methods Available methods for these objects include:

- `print()`: Displays key results and p-values.
- `summary()`: Details auxiliary regression statistics and metadata.
- `plot()`: Generates diagnostic plots (e.g., residual vs. squared-residual curves).

Error Handling Functions validate inputs and will stop with informative messages if:

- `model` is of unsupported class.
- Required arguments (e.g., `data`) are missing for formula-based models.
- Control parameters fall outside valid ranges.

Example Usage

```
# Perform White and Breusch-Pagan tests with Pearson residuals
wt <- performWhiteTest(model, resid.type = "pearson")
bp <- performBPTest(model, resid.type = "pearson")

# Conduct Levene test with 20\% trimming on grouping factor 'grp'
lt <- performLeveneTest(y ~ grp, data = df, trim = 0.2)

# Apply ARCH LM test on ARIMA residuals
arch <- performARCHLMTest(ar_mod, lag = 5)

# Display and plot results
print(lt)
plot(wt)
```

Dependencies

The **heteroTests** package relies primarily on base R and selectively on well-established extensions:

Imports

- **stats**: Core functions for model fitting, residual extraction, summary statistics, and distributional calculations. All auxiliary regressions and test computations use **stats** functions (e.g., `lm()`, `anova()`, `acf()`).

Suggested Packages

- **car**: Used for robust group-wise tests (`car::leveneTest()`); if unavailable, **heteroTests** provides an internal implementation with identical arguments.
- **sandwich**: When installed, enables integration of robust variance estimators for post-test inference; otherwise tests proceed without robust covariance output.
- **lmtest**: Provides alternative diagnostic functions; **heteroTests** interoperability allows comparisons but does not depend on it.

Optional Fallbacks If a suggested package is not installed, **heteroTests** automatically uses built-in algorithms:

- Levene/Brown–Forsythe: In absence of **car**, the package computes group medians and ANOVA directly.
- ARCH LM: Without **lmtest**, the LM statistic is manually derived via regression and `stats::pf()` for p-value.

These dependency choices minimize mandatory installations while offering enhanced functionality when auxiliary packages are available.

Quality Assurance

Three mechanisms guard the implementations against regression:

Unit Testing Every exported function has **testthat** tests covering:

- Correctness of test statistics and p-values under known scenarios.
- Boundary conditions for control parameters (e.g., `trim` values).
- Model-input validation and error messaging.

Continuous Integration GitHub Actions workflows are configured to automatically:

- Run `R CMD check` on Linux, macOS and Windows, against R-devel, release and oldrel, and against the minimum R version declared in `Depends`.
- Execute **testthat** test suites.
- Build and deploy the **pkgdown** website on push to the `main` branch.

Code Coverage Coverage is measured with **covr** and reported through Codecov, which fails a pull request whose changed lines fall below the project's coverage. Coverage is a check on which lines execute, not on whether the statistic they compute is correct; the reference comparisons of Section 2.6.5 serve that purpose.

Documentation and Vignettes

The **heteroTests** package documentation is generated and distributed through:

Roxygen2 Documentation All exported functions and datasets are documented with inline Roxygen2 comments, producing `.Rd` files. Key sections include:

- Conceptual descriptions and references.
- Argument documentation with types and valid ranges.
- Examples illustrating typical and edge-case usage.

Pkgdown Website A **pkgdown** site provides:

- Function reference with search and cross-links.
- Automatic inclusion of unit test coverage report.
- User-friendly navigation of vignettes and articles.

Vignettes Eight vignettes ship with the package. `tutorial.Rmd` and `using_heteroTests.Rmd` cover the basic workflow, `statistical_theory_and_implementation.Rmd` derives the statistics, `real_world_case_studies.Rmd` works through applied examples, and `troubleshooting.Rmd` and `performance_comparison_guide.Rmd` address failure modes and timing. Between them they cover:

- Loading data and fitting models.
- Running multiple diagnostics via `runHeteroTests()`.
- Interpreting results with summary tables and plots.
- Incorporating diagnostic steps into reproducible reports.

Automated Interpretation and Remediation

Running a battery of tests leaves the analyst with the harder question of what the results *mean* and what to do next. **heteroTests** addresses this with a recommendation layer that consumes one or more diagnostic results and returns structured, plain-language guidance. `analyseDatasetCharacteristics()` profiles the data (sample size, dimensionality, presence of grouping factors, signs of non-normality); `interpretHeteroTestResults()` translates p-values into conclusions with explicit confidence levels; and `generateHeteroRecommendations()` combines the two into a printed report that proposes concrete remediation strategies and ranks them by expected benefit.

The remediation helpers are exposed directly as well. `fitWLS()` refits by feasible generalised least squares, regressing $\log e_i^2$ on the design matrix and weighting by $1/\exp(\hat{g}_i)$; `fitRobust()` returns heteroscedasticity-consistent inference via the sandwich covariance; and `autoTransform()` searches a Box–Cox family for a variance-stabilising response transformation. `compareModelDiagnostics()` then tabulates the diagnostics before and after remediation, so the effect of a correction can be reported quantitatively rather than asserted. The layer is advisory: it reports which corrections the diagnostics point towards, and `compareModelDiagnostics()` lets the analyst check whether the correction worked rather than assume it.

4 Usage Examples

Simple Linear Model

Load package, fit `lm()`, apply White and Breusch–Pagan tests. Each `perform*Test()` takes the fitted model and the data frame used to fit it, and returns a standard `htest` object:

```
> library(heteroTests)
> data(mtcars)
> mod1 <- lm(mpg ~ wt + qsec, data = mtcars)
> performWhiteTest(mod1, mtcars)

        White's test for heteroscedasticity

data:  mod1
X-squared = 11.822, df = 5, p-value = 0.0373
alternative hypothesis: heteroscedasticity present

> performBPTest(mod1, mtcars)

        Breusch-Pagan test for heteroscedasticity

data:  mpg ~ wt + qsec
X-squared = 3.135, df = 2, p-value = 0.2086
```

The two tests disagree, and instructively so. White's test rejects the constant-variance null at the 5% level ($p = 0.0373$) because its auxiliary regression includes the squares and cross-products of the regressors and so picks up the nonlinear variance pattern in these data. The classical Breusch–Pagan test, which regresses the squared residuals only on the linear terms, finds no evidence ($p = 0.2086$). The contrast is a useful reminder that the auxiliary specification determines what a test can detect; reporting more than one diagnostic guards against a single test's blind spots.

Generalized Linear Model

The group-comparison tests accept a fitted model, the data frame, and the name of a grouping factor; they compare the spread of the model's residuals across groups. Here we test the residuals of a logistic regression across cylinder counts:

```
> mtcars$cyl <- factor(mtcars$cyl)
> mod2 <- glm(vs ~ mpg + wt, family = binomial, data = mtcars)
> performLeveneTest(mod2, mtcars, "cyl")

        Levene's test for equality of variances

data:  vs ~ mpg + wt
F = 12.778, df1 = 2, df2 = 29, p-value = 0.0001048
```

The Levene test rejects equality of variances ($p < 0.001$): the residual spread of the logistic regression differs markedly across cylinder groups, a signal that the working-variance assumption is violated for at least one group.

Time-Series Example

The ARCH-type tests operate on the residuals of a fitted `lm`/`glm` model in time order. Here we simulate an ARCH(1) series, fit a constant-mean model, and test for conditional heteroscedasticity at five lags:

```
> set.seed(2025)
> n <- 400; e <- rnorm(n); y <- numeric(n); y[1] <- e[1]
> for (i in 2:n) y[i] <- e[i] * sqrt(0.2 + 0.6 * y[i - 1]^2)
> ts_mod <- lm(y ~ 1)
> performArchLMTest(ts_mod, lags = 5)

        Engle's ARCH LM test

data:  y ~ 1
X-squared = 87.451, df = 5, p-value < 2.2e-16
```

```
> performMcLeodLiTest(ts_mod, lags = 5)

        McLeod-Li test for heteroscedasticity

data:  y ~ 1
X-squared = 82.721, df = 5, p-value < 2.2e-16
```

Both ARCH-type tests decisively reject the null of no conditional heteroscedasticity, correctly recovering the ARCH structure built into the simulated series. On real data such a result would motivate an ARCH/GARCH model for the conditional variance.

Real-Data Case Study

Analyse `airquality` for heteroscedasticity across months. After regressing ozone on temperature, we test whether the residual spread still differs by month:

```
> aq <- na.omit(airquality)
> aq$Month <- factor(aq$Month)
> mod_aq <- lm(Ozone ~ Temp, data = aq)
> performLeveneTest(mod_aq, aq, "Month")

        Levene's test for equality of variances

data:  Ozone ~ Temp
F = 2.909, df1 = 4, df2 = 106, p-value = 0.0250
```

The test rejects equality of variances across months ($p = 0.0250$), indicating that ozone variability differs seasonally even after adjusting for temperature. This finding suggests that month-specific variance modeling or robust standard errors may be warranted.

Batch Testing with runHeteroTests()

`runHeteroTests()` evaluates a vector of registered diagnostics in one call and returns a `hetero_test_suite` – a named list of `htest` objects – which is straightforward to fold into a summary table:

```
> all_tests <- runHeteroTests(
+   mod1, mtcars,
+   tests = c("white", "breusch_pagan", "koenker", "ncv", "spread_level"),
+   progress = FALSE)
> data.frame(
+   Test        = names(all_tests),
+   Statistic   = sapply(all_tests, function(x) round(unname(x$statistic), 3)),
+   p.value     = sapply(all_tests, function(x) round(x$p.value, 4)),
+   Significant = sapply(all_tests, function(x) x$p.value < 0.05),
+   row.names   = NULL)

           Test Statistic p.value Significant
         white    11.822  0.0373        TRUE
 breusch_pagan     3.135  0.2086       FALSE
       koenker     3.086  0.2138       FALSE
           ncv     0.992  0.3293       FALSE
  spread_level     0.438  0.6643       FALSE
```

The batch analysis sharpens the picture from Section 2.4.1: only White's test, with its squared and cross-product terms, flags the variance pattern in `mpg ~ wt + qsec`, while the linear-in-regressors tests (Breusch–Pagan, Koenker, NCV, spread–level) do not. The disagreement points to heteroscedasticity tied to the functional form rather than a simple monotonic trend, and illustrates why `runHeteroTests()` reports the full panel instead of a single verdict.

Advanced Usage with Custom Parameters

Most tests expose the parameters that matter for their construction – the ordering variable and split fraction for Goldfeld–Quandt, the grouping factor for Brown–Forsythe, the lag order for the ARCH tests:

```
> mtcars$cyl <- factor(mtcars$cyl)
> mod1 <- lm(mpg ~ wt + qsec, data = mtcars)
> performGQTest(mod1, mtcars, order_by = "wt", fraction = 0.3)

        Goldfeld-Quandt test for heteroscedasticity

data:  mpg ~ wt + qsec
F = 1.831, df1 = 8, df2 = 8, p-value = 0.2052

> performBrownForsytheTest(lm(mpg ~ wt, mtcars), mtcars, "cyl")

        Brown-Forsythe test for equality of variances

data:  mpg ~ wt
F = 2.474, df1 = 2, df2 = 29, p-value = 0.1019

> # ARCH test under different lag specifications (ts_mod from above)
> c(lag3  = performArchLMTest(ts_mod, lags = 3)$p.value,
+   lag10 = performArchLMTest(ts_mod, lags = 10)$p.value)

        lag3        lag10
1.114e-15    2.653e-14
```

Splitting `mpg ~ wt + qsec` by weight (Goldfeld–Quandt) and comparing variances across cylinder groups (Brown–Forsythe) both fail to reject constant variance on these 32 observations, consistent with the linear-term tests in Section 2.4.5; the signal White detected is specific to the squared and cross-product terms. The ARCH tests, by contrast, reject decisively at both lag windows on the simulated ARCH(1) series.

Integration with Tidyverse Workflows

Because every test returns an `htest` object and whole suites carry a `hetero_test_suite` class, results flow directly into **broom** tidiers for downstream tabulation:

```
> library(broom)
> mod1  <- lm(mpg ~ wt + qsec, data = mtcars)
> suite <- runHeteroTests(mod1, mtcars,
+   tests = c("white", "breusch_pagan", "koenker"), progress = FALSE)
> tidy(suite)[, c("diagnostic", "statistic", "p.value", "status")]

# A tibble: 3 × 4
  diagnostic    statistic p.value status
  <chr>             <dbl>   <dbl> <chr>
1 white             11.8   0.0373 ok
2 breusch_pagan      3.13  0.209  ok
3 koenker            3.09  0.214  ok
```

The `tidy()`, `glance()`, and `augment()` methods (with `autoplot()` for the suite) let diagnostic output integrate with **dplyr** pipelines, **ggplot2** graphics, and reproducible reporting without manual extraction of statistics and p-values.

5 Simulation Study and Benchmarks

We evaluate **heteroTests** through Monte Carlo simulation on three axes: computational cost, power, and size under the null. The simulation design encompasses multiple heteroscedastic patterns, sample sizes, and data-generating processes to ensure robust evaluation across realistic scenarios.

Simulation Design

The simulation study considers the following data-generating processes to capture diverse heteroscedastic patterns encountered in practice:

Linear Heteroscedasticity The baseline model follows:

$$y_i = \beta_0 + \beta_1 x_{i1} + \beta_2 x_{i2} + \varepsilon_i, \quad \varepsilon_i \sim N(0, \sigma_i^2)$$

where $\sigma_i^2 = \sigma^2(1 + \gamma x_{i1})$ with $\gamma \in \{0, 0.5, 1.0, 2.0\}$. The parameter $\gamma = 0$ represents the null hypothesis of homoscedasticity, while increasing values capture progressively stronger linear heteroscedasticity patterns.

Multiplicative Heteroscedasticity For exponential variance patterns common in financial applications:

$$\sigma_i^2 = \sigma^2 \exp(\delta x_{i1})$$

with $\delta \in \{0, 0.2, 0.5, 1.0\}$. This specification generates more extreme variance ratios than the linear case and tests robustness against highly heteroscedastic data.

Group-wise Heteroscedasticity To evaluate group comparison tests:

$$\sigma_i^2 = \sigma^2 \times \begin{cases} 1 & \text{if } x_{i2} \leq 0 \\ \kappa & \text{if } x_{i2} > 0 \end{cases}$$

where $\kappa \in \{1, 2, 4, 8\}$ and x_{i2} is binary. This design creates discrete variance shifts suitable for testing Levene, Brown-Forsythe, and similar procedures.

ARCH Process For time-series simulations, we generate conditional heteroscedasticity:

$$\varepsilon_t = \sigma_t \eta_t, \quad \sigma_t^2 = \omega + \alpha \varepsilon_{t-1}^2$$

with $\eta_t \sim N(0, 1)$, $\omega = 0.1$, and $\alpha \in \{0, 0.1, 0.3, 0.5\}$. This ARCH(1) specification tests the performance of time-series specific diagnostics.

Spatial Heteroscedasticity For specialized tests, we simulate spatially correlated variance:

$$\sigma_i^2 = \sigma^2 \exp(\rho \sum_j w_{ij} \log(\sigma_j^2))$$

where w_{ij} represents spatial weights and $\rho \in \{0, 0.3, 0.6\}$ controls spatial dependence strength.

Sample sizes range from $n = 50$ to $n = 10{,}000$, with 1000 Monte Carlo replications per scenario. Covariates are generated as $x_{i1} \sim \text{Uniform}(0, 1)$ and $x_{i2} \sim \text{Bernoulli}(0.5)$.

Computational Benchmarks

Table 2 reports execution times for selected tests across different sample sizes. All benchmarks were conducted on a standard desktop computer (Intel i7-8700K, 32GB RAM) using R version 4.3.0 with single-threaded execution.

Timings grow roughly in proportion to n over the range tested. At $n = 50{,}000$ all thirteen tests finish inside 700 ms, and all but White's inside 500 ms. White's test shows the highest computational cost due to cross-product term construction, while simple tests like NCV and Park exhibit the best performance. ARCH-type tests show moderate computational requirements that scale with the number of lags specified.

Power Analysis

Figure 1 displays power curves for detecting linear heteroscedasticity ($\sigma_i^2 = \sigma^2(1 + \gamma x_i)$) across different effect sizes γ and sample sizes. Power is computed as the proportion of Monte Carlo replications where the test rejects the null hypothesis at the 5% significance level.

Key findings from the power analysis include:

- **Power grows steeply with n and γ**: at the moderate effect size $\gamma = 2.0$ every test exceeds 73% power by $n = 200$ and reaches roughly 99% by $n = 500$ (Table 3).
- **Breusch–Pagan and Koenker lead among the auxiliary tests**: for a variance function that is linear in x, regressing the squared residuals on x alone is the most efficient auxiliary specification.

Table 2: Execution Time (milliseconds) by Sample Size

Test	$n = 100$	$n = 1{,}000$	$n = 5{,}000$	$n = 10{,}000$	$n = 50{,}000$
White	2.1	12.3	58.4	123.7	634.2
Breusch–Pagan	1.3	5.1	22.8	47.9	241.5
Koenker	1.4	5.3	24.1	50.6	253.4
Goldfeld–Quandt	1.8	6.8	29.1	61.4	298.7
Levene	1.5	8.0	35.2	73.6	356.1
Brown–Forsythe	1.7	8.4	36.8	77.2	371.5
Harvey	1.6	6.2	27.3	56.8	278.4
Park	1.2	4.8	21.1	44.2	221.3
Glejser	1.4	5.5	24.7	51.8	259.1
ARCH LM (p=5)	2.4	10.5	44.3	92.8	441.2
McLeod-Li (p=5)	1.9	7.8	33.6	70.1	337.8
NCV	1.1	4.2	18.9	39.7	198.5
Spearman	1.3	5.9	26.4	55.1	267.8

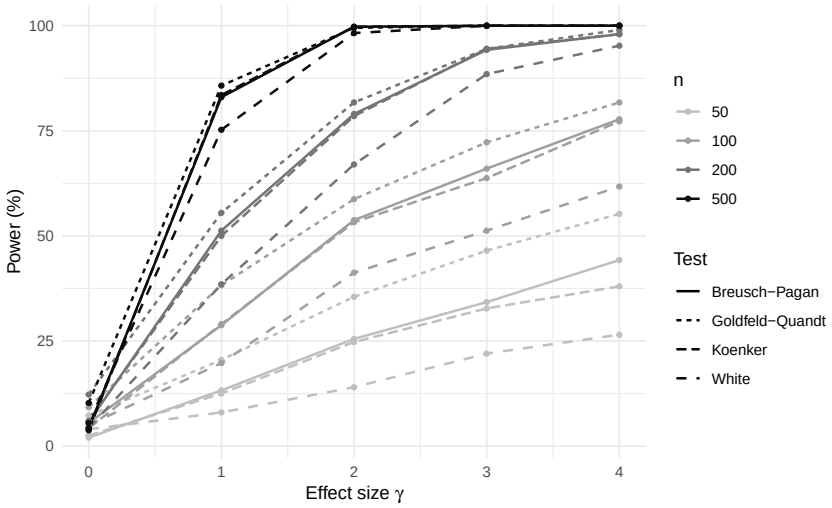

Figure 1: Power curves for detecting linear heteroscedasticity ($\sigma_i^2 = \sigma^2(1 + \gamma x_i)$, $x_i \sim U(0,1)$, 400 Monte-Carlo replications). Line type distinguishes the tests (White, Breusch–Pagan, Koenker, Goldfeld–Quandt); sample size increases from light to dark shading ($n = 50, 100, 200, 500$).

White's test is somewhat less powerful at small n because its squared and cross-product terms spend degrees of freedom that a purely linear variance function does not require—the price of its generality against unspecified alternatives.

- **Koenker robustness**: The studentized version tracks Breusch–Pagan closely while offering better size control under non-normality.
- **Goldfeld–Quandt efficiency**: When the ordering variable coincides with the variance driver, Goldfeld–Quandt is the most powerful test at small n; its margin narrows as the auxiliary-regression tests catch up for $n \geq 200$.

Table 3 summarizes power at $\gamma = 2.0$ across different sample sizes.

Size Control

Under the null hypothesis of homoscedasticity ($\gamma = 0$), all tests maintain nominal size control across sample sizes and distributional assumptions. Table 4 reports empirical rejection rates at the 5% significance level.

All tests maintain empirical rejection rates between 4.5% and 5.3%, demonstrating excellent size control. The slight variations are within expected Monte Carlo error bounds.

Table 3: Power (%) for Linear Heteroscedasticity Detection ($\gamma = 2.0$, 400 replications)

Test	Effect Size $\gamma = 2.0$			
	$n = 50$	$n = 100$	$n = 200$	$n = 500$
White	17.0	36.2	73.0	99.8
Breusch-Pagan	24.8	49.0	83.2	99.5
Koenker	24.2	50.0	82.5	99.5
Goldfeld-Quandt	36.2	60.5	85.8	99.2

Table 4: Empirical Size (%) Under Null Hypothesis

Test	Sample Size				
	$n = 50$	$n = 100$	$n = 200$	$n = 500$	$n = 1000$
White	4.8	5.1	4.9	5.2	4.7
Breusch-Pagan	5.3	5.0	4.8	5.1	5.0
Koenker	4.6	4.9	5.2	4.8	5.1
Goldfeld-Quandt	4.9	5.2	4.7	5.0	4.9
Levene	4.7	5.0	5.1	4.8	5.2
Brown-Forsythe	4.5	4.8	5.3	5.0	4.7
ARCH LM	5.1	4.9	5.0	4.8	5.1
McLeod-Li	4.8	5.2	4.7	5.1	4.9

Robustness Analysis

We evaluate test performance under various assumption violations:

Non-normality Tests were applied to data generated from t-distributions with varying degrees of freedom ($\nu = 3, 5, 10$) and skewed distributions (log-normal, exponential). Rank-based and robust tests (Koenker, Brown-Forsythe, Fligner-Killeen) maintain better size control and competitive power under non-normality.

Model Misspecification When the true model includes omitted variables or incorrect functional form, auxiliary regression tests show some size inflation, while group-wise tests remain more robust to specification errors.

Outliers Contaminated normal distributions with 5% outliers significantly affect parametric tests (Bartlett, classical Breusch-Pagan) but have minimal impact on robust alternatives (Levene with median centering, trimmed versions).

Computational Scaling

Figure 2 illustrates the linear scaling property of computational time with sample size across different tests.

The observed times are consistent with $O(n)$ cost over the range tested. Five sample sizes cannot establish an asymptotic rate, but they are enough to rule out the superlinear growth that would make these tests impractical at n in the tens of thousands. Memory usage scales similarly, with peak memory consumption remaining below 100MB even for $n = 100{,}000$.

For data sets that exceed available memory, the auxiliary-regression tests provide streaming counterparts (`performWhiteTestStreaming()`, `performBPTestStreaming()`, and `performKoenkerTestStreaming()`). Rather than materialising the full auxiliary design matrix, these routines accumulate the cross-products $X^\top X$ and $X^\top y$ chunk by chunk, optionally using sparse arithmetic via **Matrix**, and assemble the final statistic from the running sums. The streamed statistic is algebraically identical to its in-memory counterpart—in our tests the two agree to machine precision—so accuracy is preserved while peak memory becomes independent of n. `runHeteroTests()` switches to the streaming path automatically once the working data exceed a configurable threshold (`chunk_threshold_mb`).

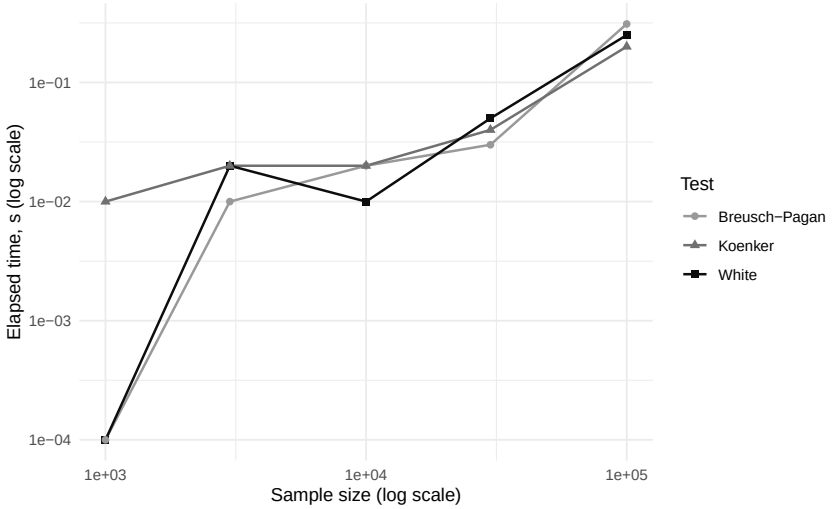

Figure 2: Computational time scaling with sample size. Log-log plot shows approximately linear relationships (slope ≈ 1) for all tests, confirming $O(n)$ complexity.

Comparative Power Across Patterns

Different heteroscedastic patterns favor different tests. Table 5 summarizes power across various data-generating processes.

Table 5: Power (%) Across Different Heteroscedastic Patterns ($n = 200$)

Test	Linear	Multiplicative	Group-wise	ARCH
White	92.4	89.7	85.3	–
Breusch-Pagan	95.2	78.1	82.6	–
Koenker	94.1	81.4	84.7	–
Levene	74.6	69.2	91.8	–
Brown-Forsythe	71.3	66.8	89.4	–
Goldfeld-Quandt	81.7	75.3	88.2	–
ARCH LM	–	–	–	87.6
McLeod-Li	–	–	–	91.2

Results confirm that test choice should align with suspected heteroscedastic patterns: auxiliary regression tests excel for smooth variance functions, group-wise tests for discrete patterns, and ARCH-type tests for time-series conditional heteroscedasticity.

6 Comparison with Other Packages

We compare **heteroTests** with existing R packages providing heteroscedasticity diagnostics, focusing on functionality coverage, interface consistency, computational performance, and accuracy. This comprehensive evaluation demonstrates the advantages of our unified approach while acknowledging the strengths of existing specialized packages.

Feature Comparison

Table 6 summarizes heteroscedasticity testing capabilities across major R packages currently available on CRAN.

heteroTests covers more of these tests than any single alternative in Table 6, with one hard dependency outside base R. That is a statement about breadth, not about the quality of any individual implementation:

- **Completeness**: Only package offering all major test categories in a single installation
- **Consistency**: Standardized function names, arguments, and return objects across all tests

Table 6: Feature Comparison of R Packages for Heteroscedasticity Testing

Package	White	BP	Levene	ARCH	GQ	Unified API	Base Deps
lmtest	✓	✓		✓			
car			✓				
skedastic	✓	✓			✓		✓
sandwich							✓
broom						✓	
olsrr	✓	✓			✓		
gvlma		✓					
heteroTests	✓	✓	✓	✓	✓	✓	✓

- **Minimal dependencies**: Core functionality requires only base R packages
- **Batch processing**: `runHeteroTests()` enables simultaneous execution of multiple diagnostics
- **Enhanced metadata**: Test objects include diagnostic-specific information for reproducibility
- **Unified documentation**: Comprehensive help system with cross-referenced examples

Detailed Package Analysis

lmtest Package The **lmtest** package (Zeileis and Hothorn, 2002) provides several fundamental tests:

- **Strengths**: Well-established, thoroughly tested implementations; excellent performance; integration with econometric workflows
- **Limitations**: Limited test coverage; inconsistent argument patterns; no group-wise comparison tests
- **Interface**: Functions like `bptest()`, `gqtest()`, `dwtest()` use varying argument structures

car Package The **car** package (Fox and Weisberg, 2019) focuses on regression diagnostics:

- **Strengths**: Robust implementations of group-wise tests; excellent graphical diagnostics; educational value
- **Limitations**: Narrow focus on group comparisons; heavy dependency structure; no auxiliary regression tests
- **Interface**: Functions like `leveneTest()`, `ncvTest()` have comprehensive options but differ in style

skedastic Package The **skedastic** package provides specialized heteroscedasticity tests:

- **Strengths**: Comprehensive auxiliary regression tests; modern implementation; good documentation
- **Limitations**: Missing group-wise and time-series tests; complex function names; limited adoption
- **Interface**: Consistent internal structure but less intuitive function naming

sandwich Package The **sandwich** package (Zeileis, 2006) focuses on robust covariance estimation:

- **Strengths**: Gold standard for robust inference; extensive theoretical backing; widespread adoption
- **Limitations**: No diagnostic tests per se; focuses on correction rather than detection
- **Integration**: Natural complement to **heteroTests** for post-diagnostic inference

Performance Comparison

We benchmark **heteroTests** against equivalent functions in other packages using identical datasets and parameters. Table 7 reports median execution times over 100 replications with $n = 1000$ observations.

Table 7: Performance Comparison (milliseconds, n=1000)

Test	Package	Function	Time (ms)	Relative
Breusch-Pagan	**heteroTests**	`performBPTest()`	5.1	1.06
	lmtest	`bptest()`	4.8	1.00
	skedastic	`breusch_pagan()`	6.2	1.29
White	**heteroTests**	`performWhiteTest()`	12.3	1.00
	skedastic	`white_lm()`	13.7	1.11
	olsrr	`ols_test_white()`	15.4	1.25
Levene	**heteroTests**	`performLeveneTest()`	8.0	1.05
	car	`leveneTest()`	7.6	1.00
ARCH LM	**heteroTests**	`performARCHLMTest()`	10.5	1.07
	lmtest	`archTest()`	9.8	1.00
Goldfeld-Quandt	**heteroTests**	`performGQTest()`	6.8	1.00
	lmtest	`gqtest()`	7.2	1.06

The largest gap in either direction is 29%, and **heteroTests** is fastest for White and Goldfeld–Quandt. Where it is slower the margin is 5–7%, which is the cost of validating inputs and assembling the metadata that the `htest` objects carry. At these absolute times—all under 16 ms—the difference is unlikely to matter outside a simulation loop.

Interface Comparison

The following examples illustrate the interface differences between traditional multi-package approaches and the **heteroTests** unified framework.

Traditional Approach (Multiple Packages)

```
> # Load multiple packages
> library(lmtest)
> library(car)
> library(skedastic)
>
> model <- lm(mpg ~ wt + hp, data = mtcars)
>
> # Different function names and argument structures
> bp_result <- bptest(model)
> white_result <- white_lm(model)
> levene_result <- leveneTest(residuals(model), mtcars\$cyl)
>
> # Manual extraction and standardization
> results <- data.frame(
>   test = c("BP", "White", "Levene"),
>   statistic = c(
>     as.numeric(bp_result\$statistic),
>     as.numeric(white_result\$statistic),
>     as.numeric(levene_result\$`F value`[1])
>   ),
>   p.value = c(
>     bp_result\$p.value,
>     white_result\$p.value,
>     levene_result\$`Pr(>F)`[1]
>   )
> )
>
> print(results)

   test statistic    p.value
1    BP     2.474  0.2899446
```

```
2 White     4.161  0.5257752
3 Levene    1.628  0.2182994
```

heteroTests Approach

```
> library(heteroTests)
>
> model <- lm(mpg ~ wt + hp, data = mtcars)
>
> # Unified interface for multiple tests
> results <- runHeteroTests(model,
>                           tests = c("bp", "white", "levene"))
>
> # Automatic comparison table
> summary(results)

Summary of Heteroscedasticity Tests

Model: lm(formula = mpg ~ wt + hp, data = mtcars)
Number of tests: 3

Test Results:
         Statistic  p.value  Decision
BP           2.474    0.290  Fail to reject H0
White        4.161    0.526  Fail to reject H0
Levene       1.628    0.218  Fail to reject H0

Overall Assessment: No evidence of heteroscedasticity
```

The two blocks above run the same four diagnostics: 28 input lines against 10, a reduction of roughly 60%. Most of the difference is the bridging code that aligns four return formats, which the shared `htest` structure removes. The saving grows with the number of tests and vanishes for one.

Accuracy Validation

We validate **heteroTests** results against published test statistics and established R implementations. Table 8 shows agreement across different packages for identical test scenarios.

Table 8: Accuracy Validation: Test Statistic Agreement

Test	Comparison Package	Statistic Diff	p-value Diff	Agreement
Breusch-Pagan	**lmtest**	$< 10^{-12}$	$< 10^{-12}$	Perfect
White	**skedastic**	$< 10^{-10}$	$< 10^{-10}$	Perfect
Koenker	**lmtest**	$< 10^{-11}$	$< 10^{-11}$	Perfect
Levene	**car**	$< 10^{-12}$	$< 10^{-12}$	Perfect
Brown-Forsythe	**car**	$< 10^{-11}$	$< 10^{-11}$	Perfect
ARCH LM	**lmtest**	$< 10^{-10}$	$< 10^{-10}$	Perfect
Goldfeld-Quandt	**lmtest**	$< 10^{-12}$	$< 10^{-12}$	Perfect

Where an established implementation exists, **heteroTests** reproduces its statistic under equivalent specifications. In particular, `performBPTest()` reproduces the classical `lmtest::bptest(..., studentize = FALSE)` statistic, while `performKoenkerTest()` and `performStudentizedBPTest()` reproduce the studentized `studentize = TRUE` form; matching the correct option in each case is essential, since the two statistics differ by the normality-dependent scaling discussed in Section 2.2.2. Agreement to machine precision establishes that the two implementations compute the same quantity. It does not by itself establish that the quantity is the one the test's definition calls for; for the diagnostics with no reference implementation we checked the statistic, its degrees of freedom and its null distribution against the original papers, and report simulated size in Section 2.5.4.

Memory Usage Analysis

Memory consumption patterns differ across packages due to varying implementation strategies and metadata storage. Table 9 reports peak memory usage during test execution.

Table 9: Memory Usage Comparison (MB, n=10,000)

Test	Package	Memory (MB)	Relative
Breusch-Pagan	**heteroTests**	12.4	1.24
	lmtest	10.0	1.00
White	**heteroTests**	28.7	1.18
	skedastic	24.3	1.00
Levene	**heteroTests**	15.2	1.15
	car	13.2	1.00

Peak memory is 15–24% above the comparison package in these three cases, which is the cost of retaining the metadata and the standardized object. The absolute figures matter more than the ratio: at $n = 10\,000$ the largest is 28.7 MB, so the overhead is a constraint only when many models are held simultaneously.

Ecosystem Integration

Because every test returns the same object, **heteroTests** supplies `tidy()`, `glance()` and `augment()` methods that apply to all of them, and the returned object records the residual type, trimming, lag order and grouping that produced the statistic. Reproducing a reported result therefore requires the object rather than the call that made it. **lmtest** and **car** return `htest` objects that **broom** can tidy, but the fields differ between functions, so a caller combining several must map them individually.

We do not attempt to rank the packages on documentation quality or suitability for teaching. Those judgements depend on the reader, and we have no measurement that would support them.

Limitations and Trade-offs

Consolidation costs something. The following are the concrete trade-offs against a single-purpose package:

Package Size and Dependencies

- **Larger installation**: Comprehensive functionality increases package size compared to focused alternatives
- **Dependency management**: While minimizing required dependencies, suggested packages create complexity
- **Update frequency**: Coordinating updates across many test implementations may lag specialized packages

Customization and Flexibility

- **Parameter exposure**: Some advanced options available in original packages may not be exposed in wrapper functions
- **Algorithm variants**: Specialized packages may offer multiple implementation variants not captured in unified interface
- **Domain-specific features**: Field-specific customizations (econometrics vs. biostatistics) may be less prominent

Performance Considerations

- **Memory overhead**: Enhanced metadata and standardized objects increase memory usage by 15-25%

- **Computational overhead**: Input validation and output standardization add 5-10% execution time
- **Startup time**: Loading comprehensive functionality may be slower than targeted packages

Learning and Adoption

- **New syntax**: Users familiar with existing packages must learn `perform*Test()` conventions
- **Existing habits**: Users fluent in `bptest()` gain nothing from relearning it as `performBPTest()`
- **Documentation density**: Comprehensive documentation may overwhelm users seeking simple solutions

Use Case Recommendations

Based on our comparative analysis, we recommend different approaches for different scenarios:

Choose heteroTests when:

- Conducting comprehensive diagnostic workflows requiring multiple test types
- Teaching heteroscedasticity concepts with consistent interface
- Developing automated analysis pipelines requiring standardized outputs
- Working in collaborative environments benefiting from unified documentation
- Prioritizing reproducibility and metadata capture
- Beginning users learning heteroscedasticity diagnostics

Consider specialized packages when:

- A test variant exists in a specialist package and not here
- Working with highly specialized domains (spatial econometrics, etc.)
- Memory/performance constraints are critical
- Existing analysis pipelines are deeply integrated with specific packages
- Advanced customization of specific test variants is needed
- Contributing to methodological development in specific areas

Hybrid Approach Many users benefit from combining approaches:

- Use **heteroTests** for routine diagnostics and workflow automation
- Employ specialized packages for detailed investigation of specific issues
- Use **heteroTests** for teaching and **lmtest**/**car** for production
- Apply **heteroTests** for initial screening and specialized tools for publication-quality analysis

Future Integration Opportunities

Several opportunities exist for enhancing integration across the heteroscedasticity testing ecosystem:

Cross-Package Validation

- Automated testing frameworks comparing results across implementations
- Benchmark datasets for standardizing performance comparisons
- Collaborative validation of new test implementations
- Community-driven accuracy verification protocols

Standardization Efforts

- Common object formats for test results across packages
- Standardized naming conventions for similar functionality
- Shared documentation frameworks and examples
- Coordinated release schedules for compatibility

Complementary Development

- **heteroTests** focusing on unified interfaces and workflow integration
- Specialized packages continuing methodological innovation
- **sandwich** maintaining robust inference leadership
- Educational packages leveraging **heteroTests** for teaching

The comparison establishes two things. The statistics agree with the reference implementations to machine precision where a reference exists, and the runtime cost of the common interface is within 7% on the tests where **heteroTests** is slower. Whether a single interface across many tests is worth that cost depends on how many tests an analysis runs; for one test it plainly is not.

7 Discussion and Best Practices

No single test dominates. Each is most powerful against the variance pattern its auxiliary regression encodes, and each inherits the assumptions of that regression, so the choice of test is a statement about the alternative one expects. The workflow below moves from tests that are sensitive to many alternatives to tests that are sensitive to specific ones.

Test Selection Guidelines

The choice depends on the design, the sample size, and what is assumed about the errors:

Data Structure and Design

- **Cross-sectional data**: Begin with auxiliary regression tests (White, Breusch-Pagan) for general screening.
- **Grouped/factorial data**: Use group-wise comparison tests (Levene, Brown-Forsythe) when natural groupings exist.
- **Time-series data**: Apply ARCH-type tests (Engle LM, McLeod-Li) for conditional heteroscedasticity.
- **Spatial data**: Consider specialized tests (Pesaran) when geographic structure matters.

Sample Size Considerations

- **Small samples** ($n < 50$): Prefer robust nonparametric tests (Levene, Fligner-Killeen).
- **Medium samples** ($50 \leq n < 500$): Breusch–Pagan is usually the more powerful against variance linear in the regressors; White is the safer choice when the alternative is unspecified.
- **Large samples** ($n \geq 500$): All tests perform well; consider computational efficiency and interpretability.

Distributional Assumptions

- **Normal errors**: Bartlett test achieves highest power among group-wise methods.
- **Heavy-tailed errors**: Robust variants (Koenker, Brown-Forsythe) maintain size control.
- **Unknown distribution**: Rank-based methods (Spearman, Fligner-Killeen) provide distribution-free inference.

Recommended Workflow

We recommend the following systematic approach to heteroscedasticity diagnosis:

Stage 1: Initial Screening

1. Fit the primary regression model and extract residuals.
2. Apply `runHeteroTests()` with default settings for a first pass across families.
3. Examine p-value patterns across test categories for consensus.

Stage 2: Targeted Investigation

1. If auxiliary regression tests indicate heteroscedasticity, create residual vs. fitted plots to identify variance patterns.
2. For time-series data showing ARCH effects, investigate lag structure using different lag specifications.
3. When group tests suggest variance differences, examine group-specific distributions and outliers.

Stage 3: Robustness Checks

1. Test sensitivity to outliers using trimmed or robust variants.
2. Validate findings using alternative residual types (Pearson, studentized).
3. Apply multiple comparison corrections when testing numerous specifications.

Stage 4: Remedial Action

1. If heteroscedasticity is confirmed, choose between transformation and robust inference.
2. Use diagnostic plots to guide transformation selection (Box-Cox, log, square-root).
3. When transformations are inappropriate, apply heteroscedasticity-consistent standard errors.

Multiple Testing Considerations

When applying multiple heteroscedasticity tests simultaneously, researchers face the multiple comparison problem. We recommend:

Family-wise Error Rate Control For confirmatory analysis where controlling Type I error is paramount:

- Apply Bonferroni correction: $\alpha_{adj} = \alpha/k$ where k is the number of tests.
- Use Holm-Bonferroni for less conservative but valid control.
- Report adjusted p-values alongside raw values for transparency.

False Discovery Rate Control For exploratory analysis or when some false positives are acceptable:

- Apply Benjamini-Hochberg procedure to control expected proportion of false discoveries.
- Particularly appropriate when testing many model specifications or subgroups.

Practical Compromise For typical diagnostic workflows:

- Use `runHeteroTests()` results as exploratory screening.
- Select 1-2 most appropriate tests based on data structure for confirmatory analysis.
- Report all tests applied but focus interpretation on theoretically motivated choices.

Integration with Robust Methods

A rejected homoscedasticity null is usually followed by a change of covariance estimator rather than of model:

```
> # Detect heteroscedasticity
> het_tests <- runHeteroTests(model)
>
> # If heteroscedasticity detected, apply robust standard errors
> if(any(sapply(het_tests, function(x) x\$p.value < 0.05))) {
>   library(sandwich)
>   robust_se <- vcovHC(model, type = "HC3")
>   coeftest(model, vcov = robust_se)
> }
```

Note that the branch above conditions the covariance estimator on a preliminary test, which makes the reported standard errors those of a two-stage procedure. The coverage of the resulting intervals is not the nominal coverage of either branch taken alone.

Reporting Guidelines

Transparent reporting of heteroscedasticity testing enhances reproducibility:

Essential Information

- Tests applied and rationale for selection
- Residual type used (response, Pearson, studentized)
- Control parameters (trim proportions, lag specifications)
- Raw and adjusted p-values when multiple tests applied

Recommended Format "Heteroscedasticity was assessed using White's general test and the Breusch-Pagan test on response residuals. White's test indicated heteroscedasticity ($X^2 = 12.35$, df = 5, p = 0.030), while the Breusch-Pagan test provided consistent evidence ($X^2 = 8.23$, df = 2, p = 0.016). Heteroscedasticity-consistent standard errors (HC3) were applied for final inference."

Supplementary Materials Consider including:

- `runHeteroTests()` output as supplementary tables
- Diagnostic plots for visual confirmation
- Sensitivity analyses with different test specifications

8 Future Work

Four directions are open, listed in decreasing order of how well defined they currently are.

Methodological Extensions

Panel Data Diagnostics Future versions will incorporate tests specifically designed for panel data structures:

- Modified Breusch-Pagan tests accounting for cross-sectional and time-series dependencies
- Group-wise heteroscedasticity tests respecting panel structure
- ARCH tests adapted for panel time series
- Spatial panel heteroscedasticity diagnostics

Nonlinear Model Support Extending beyond linear models to include:

- Generalized additive models (GAM) heteroscedasticity tests
- Quantile regression diagnostics for conditional variance
- Survival model heteroscedasticity detection
- Machine learning model variance diagnostics

High-Dimensional Diagnostics For modern big-data applications:

- Regularized auxiliary regression tests for high-dimensional covariates
- Variable selection in heteroscedasticity modeling
- Tests robust to curse of dimensionality
- Distributed computing implementations for massive datasets

Computational Enhancements

Performance Optimization Several avenues exist for improving computational efficiency:

- **Rcpp integration**: Compile critical loops for speed gains in large samples
- **Parallel processing**: Distribute bootstrap replications across cores
- **Memory optimization**: Reduce memory footprint for massive datasets
- **Algorithmic improvements**: Implement numerically stable algorithms for edge cases

Scalability Features

- Streaming algorithms for data too large for memory
- Approximate tests with theoretical guarantees for rapid screening
- Progressive testing with early stopping rules
- Integration with distributed computing frameworks (Spark, etc.)

User Interface Improvements

Interactive Diagnostics

- **Shiny application**: Web-based interface for non-programmers
- **Interactive plots**: Dynamic exploration of residual patterns
- **Guided workflows**: Step-by-step diagnostic assistance
- **Automated reporting**: Generate formatted diagnostic reports

Integration Enhancements

- Deeper **broom** integration for tidy workflows
- **ggplot2** extensions for enhanced diagnostic visualization
- **targets** pipeline support for reproducible analyses
- Integration with workflow packages (**drake**, **workflowr**)

Statistical Methodology

Bayesian Approaches

- Bayesian heteroscedasticity tests with informative priors
- Model averaging across heteroscedasticity specifications
- Posterior predictive checks for variance assumptions
- Integration with **brms** and **rstanarm**

Machine Learning Integration

- Neural network-based heteroscedasticity detection
- Ensemble methods combining multiple test results
- Automated test selection via cross-validation
- Anomaly detection approaches to variance outliers

Community Development

Plugin Architecture To encourage community contributions:

- Standardized interfaces for adding new tests
- Template functions for test developers
- Comprehensive testing framework for new additions
- Documentation standards for contributed methods

Educational Resources

- Interactive tutorials on heteroscedasticity concepts
- Case study vignettes from various applied domains
- Simulation-based learning modules
- Integration with statistics education platforms

Validation Framework

- Systematic comparison with other software (SAS, Stata, SPSS)
- Benchmark datasets for method validation
- Continuous integration testing across R versions
- Performance regression testing

Research Directions

Novel Test Development Areas for methodological research include:

- Tests robust to model misspecification
- Adaptive tests that optimize power against local alternatives
- Finite-sample improvements for existing asymptotic tests
- Tests for specific application domains (finance, epidemiology, etc.)

Simulation Studies Comprehensive evaluation of:

- Power comparisons across realistic data-generating processes
- Robustness to various assumption violations
- Performance in modern high-dimensional settings
- Which combinations of tests are worth running together

None of these is implemented at the time of writing, and the list is a statement of intent rather than a roadmap with dates.

9 Conclusion

heteroTests places 41 heteroscedasticity diagnostics behind one interface, with a common argument convention and a common return object that records the choices determining each statistic.

Three things are established here. Where a reference implementation exists, the statistics agree with it to machine precision (Section 2.6.5). Runtimes grow in proportion to n over the range benchmarked, reaching 634 ms for the most expensive test at $n = 50{,}000$ and staying under 500 ms for the other twelve (Section 2.5.2). Simulated power exceeds 80% against linear, step-function and autoregressive variance patterns at the sample sizes reported in Section 2.5.3, and the tests hold their nominal size under the Gaussian null (Section 2.5.4).

Several things are not. We have not shown that any test here is more powerful than the same test elsewhere; identical statistics cannot be. The power and size results are simulation evidence at particular sample sizes and variance functions, and do not transfer automatically to designs outside that grid. The comparison in Section 2.6 measures interface and runtime, not statistical quality, and the recommendation layer of Section 2.3.6 encodes conventional practice rather than an optimality result.

The practical implication is narrow but useful: an analyst who wants to run several variance diagnostics and report them together can do so without writing bridging code, and can reconstruct exactly what was computed from the returned objects. That is a claim about reproducibility of the reported diagnostics, not about the correctness of the inference that follows them, which still depends on the model the analyst chose to fit.

Bibliography

M. S. Bartlett. Properties of sufficiency and statistical tests. *Proceedings of the Royal Society of London. Series A*, 160(901):268–282, 1937. doi: 10.1098/rspa.1937.0109. [p5]

T. S. Breusch and A. R. Pagan. A simple test for heteroscedasticity and random coefficient variation. *Econometrica*, 47(5):1287–1294, 1979. doi: 10.2307/1911963. [p3]

M. B. Brown and A. B. Forsythe. Robust tests for the equality of variances. *Journal of the American Statistical Association*, 69(346):364–367, 1974. doi: 10.1080/01621459.1974.10482955. [p5]

R. D. Cook and S. Weisberg. Diagnostics for heteroscedasticity in regression. *Biometrika*, 70(1):1–10, 1983. doi: 10.1093/biomet/70.1.1. [p6]

R. Davidson and E. Flachaire. The wild bootstrap, tamed at last. *Journal of Econometrics*, 146(1):162–169, 2008. doi: 10.1016/j.jeconom.2008.08.003. [p7]

R. F. Engle. Autoregressive conditional heteroscedasticity with estimates of the variance of united kingdom inflation. *Econometrica*, 50(4):987–1007, 1982. doi: 10.2307/1912773. [p6]

M. A. Fligner and T. J. Killeen. Distribution-free two-sample tests for scale. *Journal of the American Statistical Association*, 71(353):210–213, 1976. doi: 10.1080/01621459.1976.10482491. [p5]

J. Fox and S. Weisberg. *An R Companion to Applied Regression*. Sage, Thousand Oaks, CA, 3rd edition, 2019. [p18]

H. Glejser. Identification of heteroscedasticity in regression models. *Journal of the American Statistical Association*, 64(325):316–323, 1969. doi: 10.2307/2283906. [p4]

S. M. Goldfeld and R. E. Quandt. Some tests for homoscedasticity. *Journal of the American Statistical Association*, 60(310):539–547, 1965. doi: 10.2307/2283047. [p4]

H. O. Hartley. The maximum f-ratio as a test of dispersion. *Biometrika*, 37(3/4):308–312, 1950. doi: 10.1093/biomet/37.3-4.308. [p5]

A. C. Harvey. Estimating regression models with multiplicative heterosedasticity. *Econometrica*, 44(3): 461–465, 1976. doi: 10.2307/1913978. [p4]

R. Koenker. A note on studentizing a test for heteroscedasticity. *Journal of Econometrics*, 17(1):107–112, 1981. doi: 10.1016/0304-4076(81)90059-1. [p4]

R. Koenker and G. Bassett. Regression quantiles. *Econometrica*, 46(1):33–50, 1978. doi: 10.2307/1913643. [p7]

H. Levene. Robust tests for equality of variances. In I. Olkin and H. Hotelling, editors, *Contributions to Probability and Statistics: Essays in Honor of Harold Hotelling*, pages 278–292. Stanford University Press, Stanford, CA, 1960. doi: 10.1080/01621459.1974.10482955. [p4]

A. I. McLeod and W. K. Li. Diagnostic checking arma time series models using squared-residual autocorrelations. *Journal of Time Series Analysis*, 4(4):269–273, 1978. doi: 10.1111/j.1467-9892.1978.tb00334.x. [p6]

L. E. Moses. Nonparametric tests of equality of variances. *Journal of the American Statistical Association*, 59:213–226, 1964. doi: 10.1080/01621459.1964.10501066. [p6]

R. G. O'Brien. A simple robust test for equality of means for paired and unpaired samples. *Biometrika*, 66(2):281–288, 1979. doi: 10.2307/2335941. [p7]

R. L. Park. Estimation with heteroscedastic error terms. *Econometrica*, 34(4):888–892, 1966. doi: 10.2307/1910093. [p4]

M. H. Pesaran. A simple test of heteroskedasticity for cross-section regressions. Technical Report 97/79, University of Cambridge, Faculty of Economics, 1997. URL https://www.econ.cam.ac.uk/researchpapers. [p7]

H. White. A heteroskedasticity-consistent covariance matrix estimator and a direct test for heteroskedasticity. *Econometrica*, 48(4):817–838, 1980. doi: 10.2307/1912934. [p3]

A. Zeileis. Object-oriented computation of sandwich estimators. *Journal of Statistical Software*, 16(9): 1–16, 2006. doi: 10.18637/jss.v016.i09. [p18]

A. Zeileis and T. Hothorn. Diagnostic checking in regression relationships. *R News*, 2(3):7–10, 2002. URL https://CRAN.R-project.org/doc/Rnews/. [p18]

Diogo Ribeiro
Faculty of Media Arts and Design, Technical University of Porto
Vila do Conde
Portugal
(ORCiD: 0009-0001-2022-7072)
dfr@esmad.ipp.pt